

In-depth Imagerie

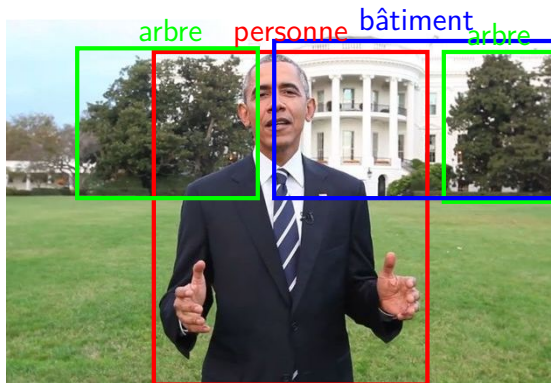
Détection d'objet

Axel Carlier

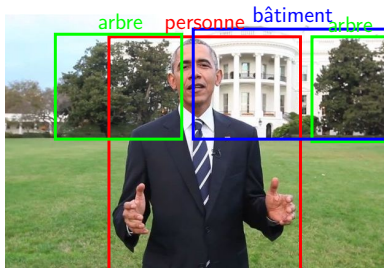
ISAE-SUPAERO, Département d'ingénierie des systèmes complexes (DISC)

`axel.carlier@isae-supaero.fr`

2026



Objectif : délimiter la position d'objets multiples, avec éventuellement plusieurs instances, toujours à l'aide de boîtes englobantes.

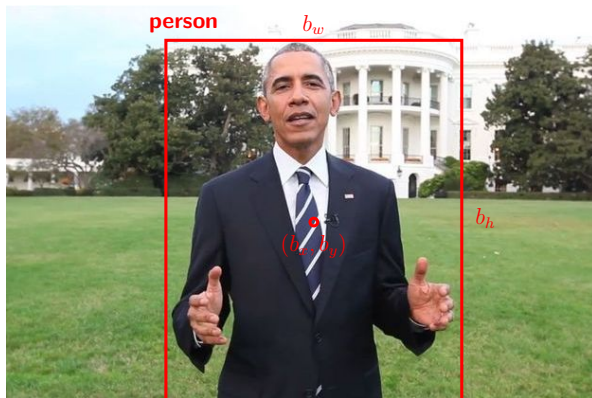


Difficultés de la détection d'objet :

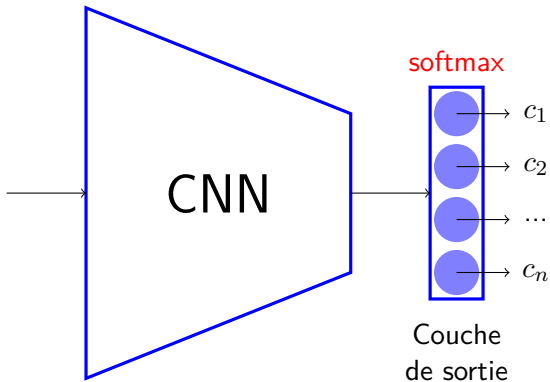
- On ne connaît pas à l'avance le nombre d'objets présents sur l'image
- Comment encoder une boîte englobante ?
- Comment évaluer l'algorithme ?

- 1 Localisation d'objet
- 2 Evaluation
- 3 Détection d'objet
 - Versions "naïves"
 - La famille des R-CNN
 - La famille des YOLO

Classification et localisation conjointes

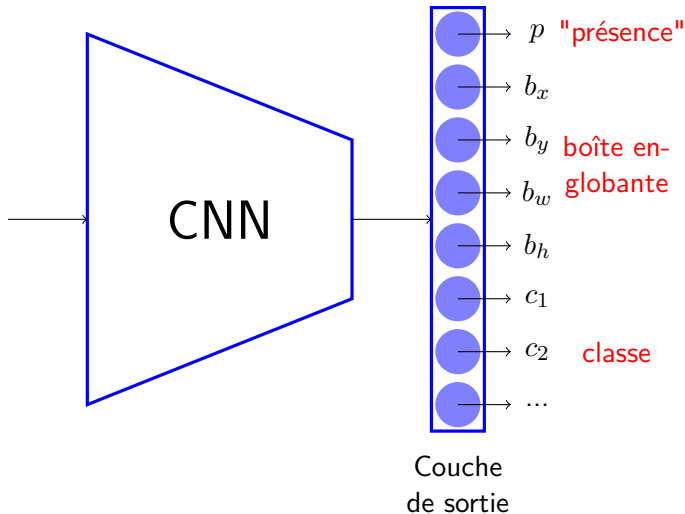


On commence par un problème simplifié : classifier et localiser **un seul objet** par image.



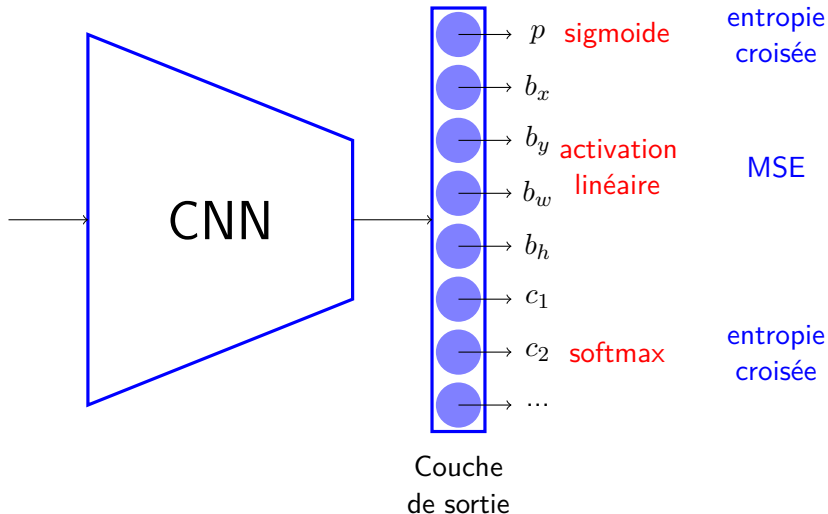
Nous devons ajouter une information spatiale sur l'objet classifié.

Classification et localisation conjointes



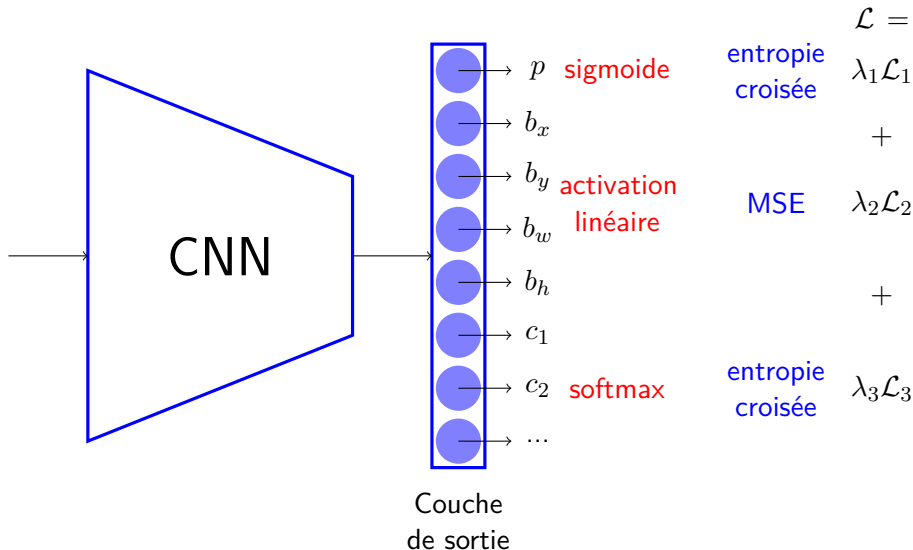
Nous pouvons rajouter les coordonnées de la boîte englobante autour de l'objet.

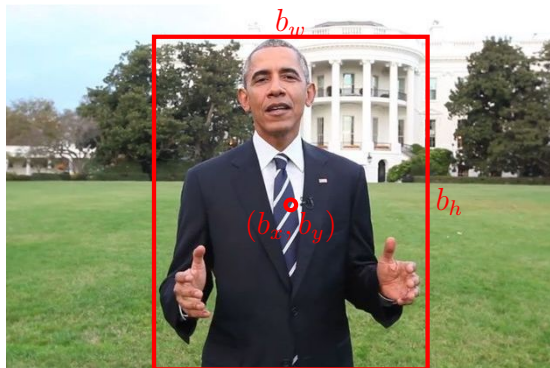
Quelle fonction de coût ?



Entraînement du modèle

Quelle fonction de coût ?

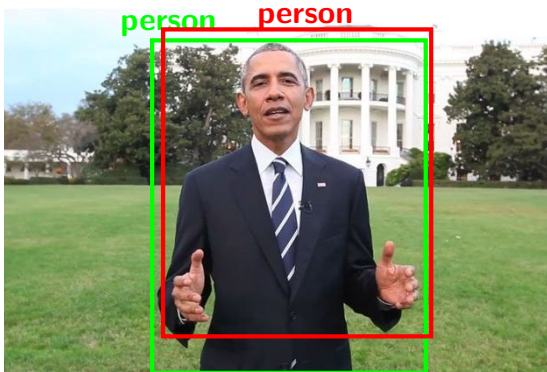




$$\hat{y} = \begin{pmatrix} 1 \\ 0.52 \\ 0.46 \\ 0.5 \\ 0.91 \\ 0 \\ \dots \\ 1 \\ .. \\ 0 \end{pmatrix}$$

- 1 Localisation d'objet
- 2 Evaluation
- 3 Détection d'objet
 - Versions "naïves"
 - La famille des R-CNN
 - La famille des YOLO

Qu'est-ce qu'une bonne détection ? On cherche à comparer une **prédiction** à une **vérité-terrain**.

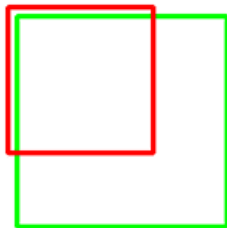


Une bonne prédiction doit être bien localisée **et** bien reconnue.

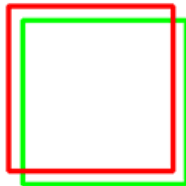
L'IoU (*Intersection over Union*) est une mesure classique de la qualité de la localisation :

$$\text{IoU}(R_1, R_2) = \frac{\mathcal{A}(R_1 \cap R_2)}{\mathcal{A}(R_1 \cup R_2)}$$

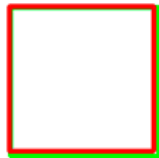
IoU: 0.4034



IoU: 0.7330

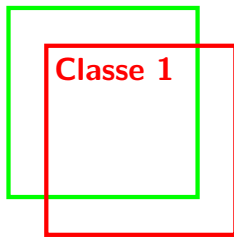


IoU: 0.9264



Prédiction et Vérité-Terrain.

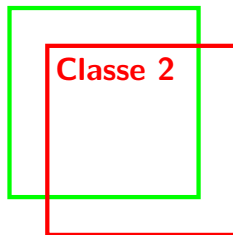
Classe 1



Classe 1 = Classe 1 et $\text{IoU} > 0.5$
On compte un Vrai Positif pour la classe 1.

Prédiction et Vérité-Terrain.

Classe 1

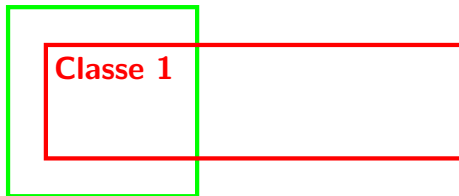


Classe 1 $\neq$ Classe 2 et $\text{IoU} > 0.5$

On compte un Faux Positif pour la classe 2 et un Faux Négatif pour la classe 1.

Prédiction et Vérité-Terrain.

Classe 1



Classe 1 = Classe 1 et $\text{IoU} < 0.5$

On compte un Faux Positif pour la classe 1 et un Faux Négatif pour la classe 1.

On définit les deux métriques complémentaires Précision (*Precision* P) et Rappel (*Recall* R) :

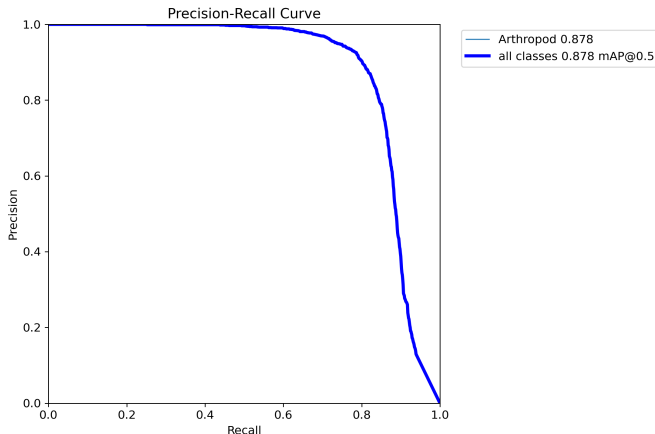
$$P = \frac{TP}{TP + FP}$$

$$R = \frac{TP}{TP + FN}$$

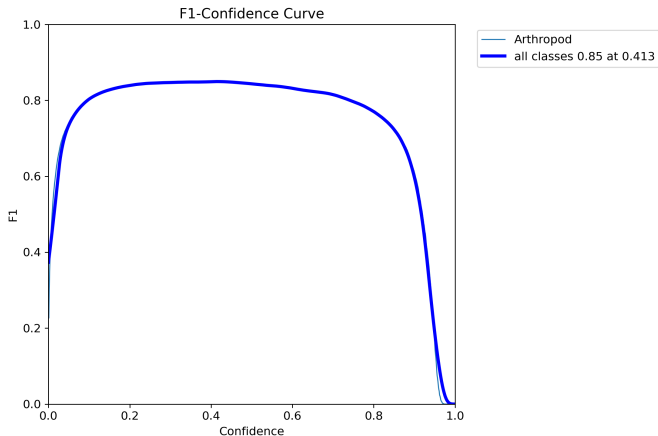
La Précision désigne, parmi les objets identifiés par le détecteur, le ratio d'objets corrects.

Le Rappel désigne, parmi les objets qu'il aurait fallu identifier, le ratio réellement identifié par le détecteur.

Courbe Precision-Rappel pour un détecteur



On fait varier le seuil de confiance en deçà duquel on élimine les détections du modèle, et on calcule à chaque fois la précision et le rappel obtenus.

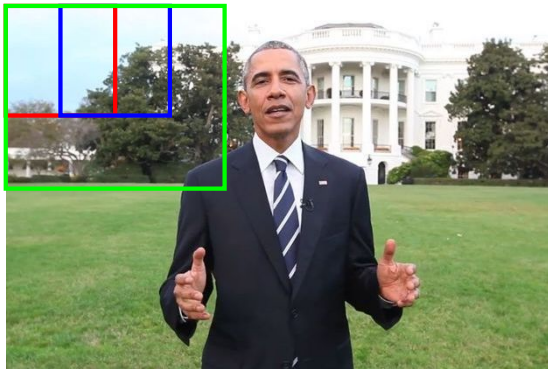


$$F1 = \frac{2}{\frac{1}{P} + \frac{1}{R}}$$

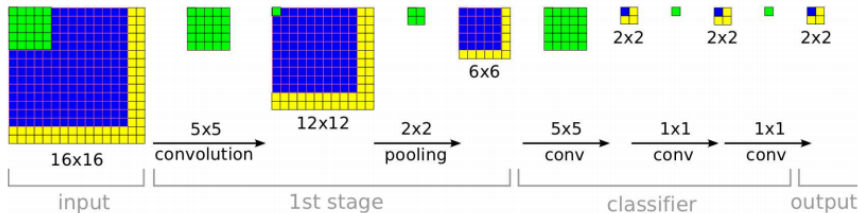
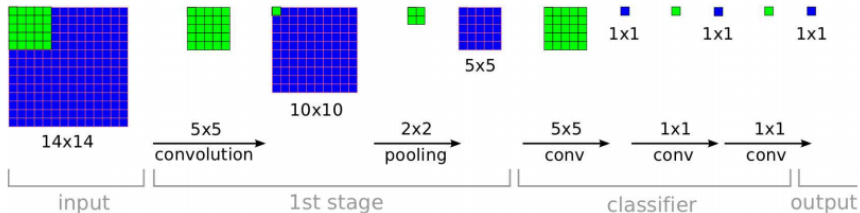
- 1 Localisation d'objet
- 2 Evaluation
- 3 Détection d'objet
 - Versions "naïves"
 - La famille des R-CNN
 - La famille des YOLO

- 1 Localisation d'objet
- 2 Evaluation
- 3 Détection d'objet
 - Versions "naïves"
 - La famille des R-CNN
 - La famille des YOLO

Une première idée : utiliser une fenêtre glissante multi-échelle pour localiser sur chaque sous-partie de l'image les objets présents.



Fenêtre glissante convolutionnelle

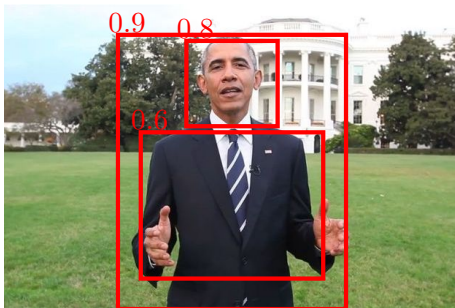


[Sermanet et al.] OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks

Fenêtre glissante convolutionnelle



[Sermanet et al.] OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks



On définit deux seuils : IoU_threshold et $\text{confidence_threshold}$.

- 1 Suppression des boîtes avec un indice de confiance inférieur à $\text{confidence_threshold}$.
- 2 Sélection parmi les boîtes restantes de la boîte b_{max} avec le plus haut indice de confiance.
- 3 Suppression de toutes les boîtes ayant une intersection sur union supérieure à IoU_threshold avec b_{max} .

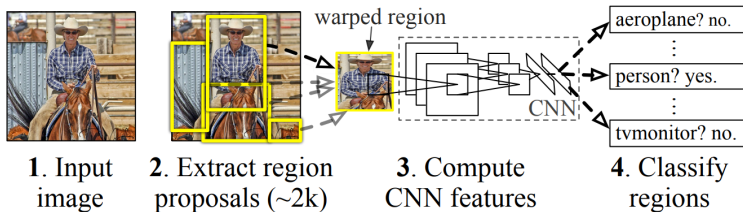
- 1 Localisation d'objet
- 2 Evaluation
- 3 Détection d'objet
 - Versions "naïves"
 - La famille des R-CNN
 - La famille des YOLO

Pour éviter les fenêtres glissantes : génération de régions candidates.



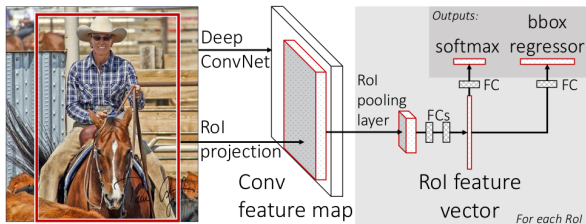
[Uijlings et al.] Selective Search for Object Recognition

Génération des régions candidates et classification de ces régions.



Inconvénient de cette méthode : nécessité de faire de nombreuses prédictions par un CNN pour chaque image. **Très peu optimisé !**

[Girshick et al.] Rich feature hierarchies for accurate object detection and semantic segmentation



La partie droite est entraînée séparément, par-dessus la base convolutive, à l'aide d'une fonction de coût hybride :

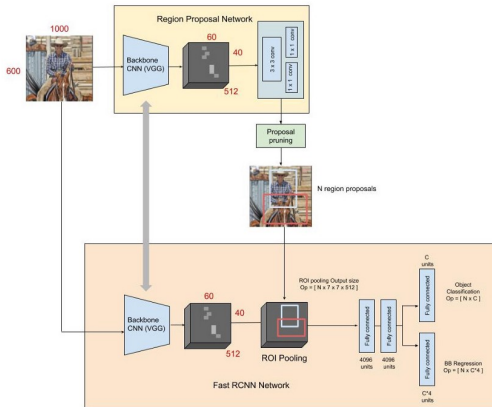
$$L(p, u, t^u, v) = L_{\text{cls}}(p, u) + \lambda[u \geq 1]L_{\text{loc}}(t^u, v), \quad (1)$$

où

$$L_{\text{loc}}(t^u, v) = \sum_{i \in \{x, y, w, h\}} \text{smooth}_{L_1}(t_i^u - v_i),$$

$$\text{smooth}_{L_1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise,} \end{cases}$$

Un même réseau (poids partagés) pour générer des régions candidates et évaluer si elles contiennent un objet (plus rapide mais pas temps réel !)



[Ren et al.] Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks

Detectron2 :

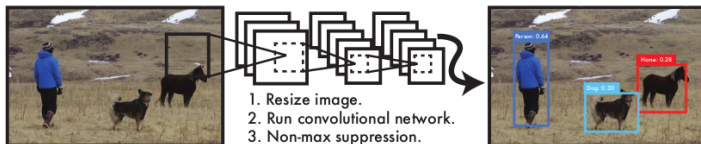
<https://github.com/facebookresearch/detectron2>

Faster R-CNN:

Name	lr sched	train time (s/iter)	inference time (s/im)	train mem (GB)	box AP	model id	download
R50-C4	1x	0.551	0.102	4.8	35.7	137257644	model metrics
R50-DC5	1x	0.380	0.068	5.0	37.3	137847829	model metrics
R50-FPN	1x	0.210	0.038	3.0	37.9	137257794	model metrics
R50-C4	3x	0.543	0.104	4.8	38.4	137849393	model metrics
R50-DC5	3x	0.378	0.070	5.0	39.0	137849425	model metrics
R50-FPN	3x	0.209	0.038	3.0	40.2	137849458	model metrics
R101-C4	3x	0.619	0.139	5.9	41.1	138204752	model metrics
R101-DC5	3x	0.452	0.086	6.1	40.6	138204841	model metrics
R101-FPN	3x	0.286	0.051	4.1	42.0	137851257	model metrics
X101-FPN	3x	0.638	0.098	6.7	43.0	139173657	model metrics

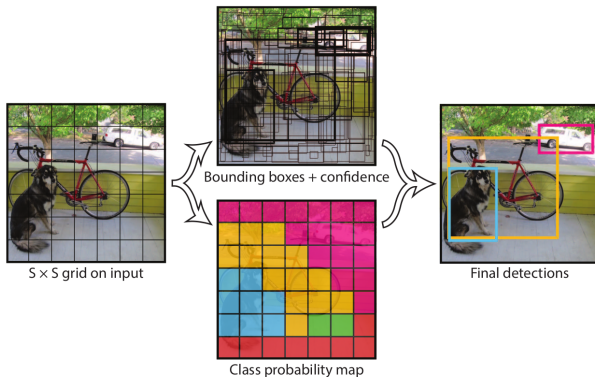
- 1 Localisation d'objet
- 2 Evaluation
- 3 Détection d'objet
 - Versions "naïves"
 - La famille des R-CNN
 - La famille des YOLO

You Only Look Once = une seule inférence du réseau permet de générer toutes les boîtes en même temps.



Problème : Comment faire en sorte de générer un nombre de boîtes adapté (et potentiellement différent) pour chaque image ?

Solution : Générer trop de boîtes, puis filtrer !



YOLO divise l'image en $S \times S$ cellules, et prédit B boîtes englobantes pour chaque cellule. Une boîte englobante est définie par 4 coordonnées et un indice de confiance.

[Redmon et al.] You Only Look Once: Unified, Real-Time Object Detection

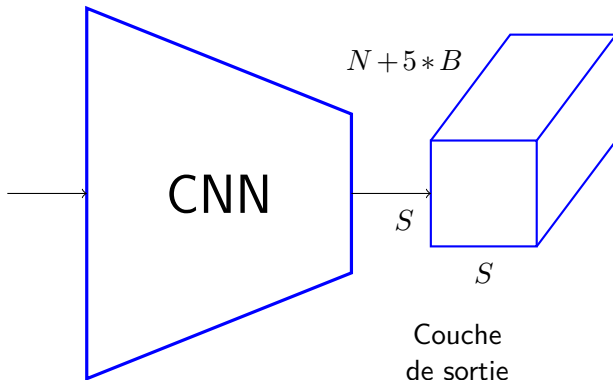
L'indice de confiance $c_{\mathcal{B}_p}$ d'une boîte englobante prédite $\mathcal{B}_p$ est défini comme suit :

$$c_{\mathcal{B}_p} = \mathbb{1}^{\text{obj}} \text{ IOU}(\mathcal{B}_p, \mathcal{B}_{GT})$$

Ainsi, s'il n'y a pas d'objet dans la cellule contenant $\mathcal{B}_p$, $c_{\mathcal{B}_p} = 0$.

S'il y a un objet dans la cellule, alors $c_{\mathcal{B}_p} = \text{IOU}(\mathcal{B}_p, \mathcal{B}_{GT})$.

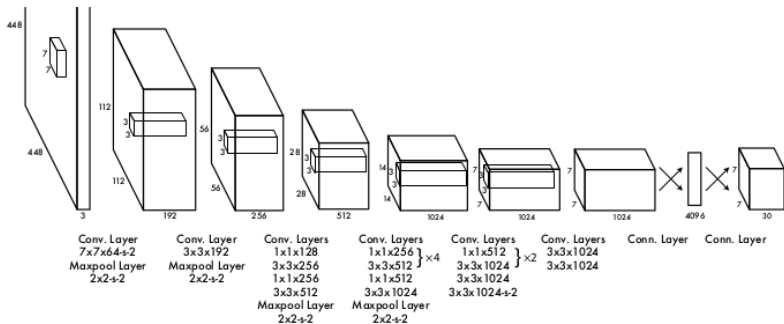
On entraîne ainsi le modèle à détecter des objets mais aussi à prédire avec quelle précision il a détecté un objet.



où :

- S est la taille de la grille (sur PASCAL VOC, $S = 7$)
- B est le nombre d'objets détectés par cellule (PASCAL : $B = 2$)
- N est le nombre de classes (PASCAL : $N = 20$)

[Redmon et al.] You Only Look Once: Unified, Real-Time Object Detection



Architecture librement inspirée d'Inception !

YOLO: fonction de perte

$$\begin{aligned} & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \quad \text{coord loss} \\ & + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\ & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\ & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \quad \text{no box loss} \\ & + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \quad \text{box loss} \end{aligned} \quad (3)$$

Une erreur quadratique moyenne sur tous les termes !

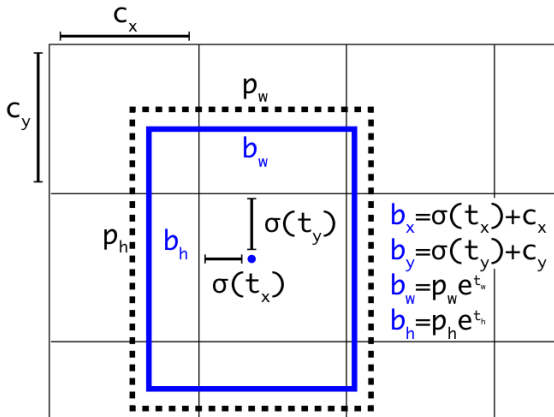
Le problème des petits objets



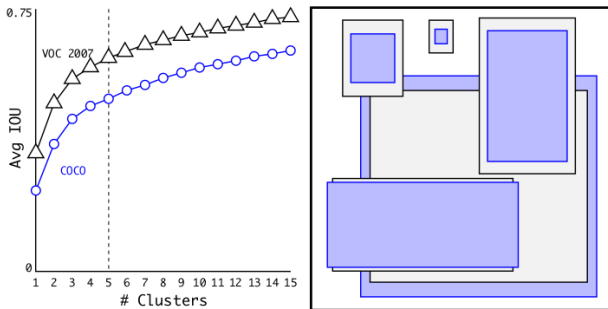
De nombreuses améliorations sont réalisées, par exemple :

- Utilisation de "modèles" de boîtes englobantes (*anchor boxes*).
- Modification de la manière d'encoder les boîtes englobantes.
- Prédiction de probabilités de classe différentes pour chaque boîte englobante
- Entraînement multi-échelle.
- Optimisations sur l'architecture Darknet.

[Redmon et al.] YOLO9000 : Better, Faster, Stronger

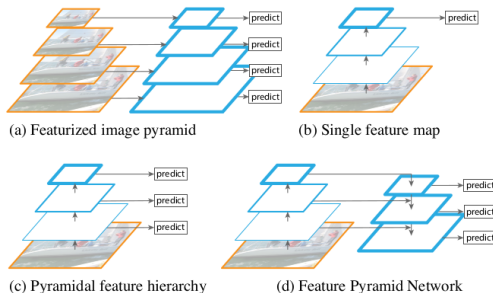


t_x, t_y, t_w, t_h peuvent prendre n'importe quelle valeur réelle, le problème est plus facile à apprendre pour le modèle.



Algorithme des k -moyennes en utilisant l'IOU pour mesurer la similarité entre les boîtes.

- Darknet-53, un ResNet-like !
- Remplacement de softmax par sigmoid pour la prédiction des classes (utile pour certains datasets où les classes ne sont pas mutuellement exclusives)
- L'utilisation de Feature Pyramid Networks résoud en grande partie le problème des petits objets.



YOLOv4 consists of:

- Backbone: CSPDarknet53 [81]
- Neck: SPP [25], PAN [49]
- Head: YOLOv3 [63]

YOLO v4 uses:

- Bag of Freebies (BoF) for backbone: CutMix and Mosaic data augmentation, DropBlock regularization, Class label smoothing
- Bag of Specials (BoS) for backbone: Mish activation, Cross-stage partial connections (CSP), Multi-input weighted residual connections (MiWRC)
- Bag of Freebies (BoF) for detector: CIoU-loss, CmBN, DropBlock regularization, Mosaic data augmentation, Self-Adversarial Training, Eliminate grid sensitivity, Using multiple anchors for a single ground truth, Cosine annealing scheduler [52], Optimal hyperparameters, Random training shapes
- Bag of Specials (BoS) for detector: Mish activation, SPP-block, SAM-block, PAN path-aggregation block, DIoU-NMS

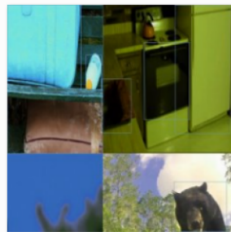
Un nouveau type d'augmentation :



aug_-319215602_0_-238783579.jpg



aug_-1271888501_0_-749611674.jpg



aug_1462167959_0_-1659206634.jpg



aug_1474493600_0_-45389312.jpg



aug_1715045541_0_603913529.jpg



aug_1779424844_0_-589696888.jpg

- ① **YOLOv5**: <https://github.com/ultralytics/yolov5>
 - ▶ **YOLOv7** Wang et al. 2021 : You Only Learn One Representation: Unified Network for Multiple Tasks)
 - ▶ **YOLOv7** Wang et al. 2022: YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors
- ② **YOLOX** Ge et al. 2021 YOLOX: Exceeding YOLO Series in 2021
- ③ **YOLOv6** Li et al. 2022 YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications

YOLOv8 :

<https://github.com/ultralytics/yolov8>

YOLOv11 :

<https://github.com/ultralytics/ultralytics>

 Performance

[Détection \(COCO\)](#)
[Segmentation \(COCO\)](#)
[Classification \(ImageNet\)](#)
[Pose \(COCO\)](#)
[OBB \(DOTAv1\)](#)

Voir [Detection Docs](#) pour des exemples d'utilisation de ces modèles formés sur [COCO](#), qui comprennent 80 classes préformées.

Modèle	taille (pixels)	mAPval 50-95	Vitesse CPU ONNX (ms)	Vitesse T4 TensorRT10 (ms)	params (M)	FLOPs (B)
YOLO11n	640	39.5	56.1 ± 0.8	1.5 ± 0.0	2.6	6.5
YOLO11s	640	47.0	90.0 ± 1.2	2.5 ± 0.0	9.4	21.5
YOLO11m	640	51.5	183.2 ± 2.0	4.7 ± 0.1	20.1	68.0
YOLO11l	640	53.4	238.6 ± 1.4	6.2 ± 0.1	25.3	86.9
YOLO11x	640	54.7	462.8 ± 6.7	11.3 ± 0.2	56.9	194.9

Institut Supérieur de l'Aéronautique et de l'Espace

10 avenue Édouard Belin – BP 54032

31055 Toulouse Cedex 4 – France

Phone: +33 5 61 33 80 80

www.isae-superaero.fr